

Lab Session 14

Introduction to Web Project Deployment and Maintenance Using GitHub Pages

1. Introduction

In modern software engineering, developing an application is only one stage of the **Software Development Life Cycle (SDLC)**. After the software has been designed, implemented, and tested, it must be **deployed** so that users can access and use it. Deployment refers to the process of making an application available in a real-world environment.

Deployment is a critical stage because it transforms the software from a local development project into a **publicly accessible system**. Without deployment, software cannot reach end users or provide practical value.

Once the application is deployed, developers must perform **software maintenance**. Maintenance involves updating the system to fix errors, improve functionality, enhance performance, and adapt to new requirements.

In web development, deployment typically means **hosting a website on a server** so that it becomes accessible through the internet using a public URL.

One of the easiest platforms for deploying static websites is **GitHub** using its hosting service **GitHub Pages**. GitHub Pages allows developers to publish websites directly from a repository without requiring a separate hosting provider.

In this lab, students will learn how to deploy an **existing web project** using GitHub Pages and then perform updates to simulate the **maintenance phase of SDLC**.

2. Learning Objectives

After completing this lab, students will be able to:

- Understand the concept of **software deployment**
- Understand the role of **software maintenance** in SDLC
- Understand the importance of **version control in deployment**
- Upload an existing project to GitHub
- Deploy a web application using GitHub Pages
- Access a deployed website through a public URL
- Perform updates and maintenance on a deployed application

3. Software and Tools Required

The following tools will be used in this lab:

Tool	Purpose
Git	Version control system used to track code changes
GitHub	Online platform for hosting repositories
GitHub Pages	Service used to deploy static websites
Visual Studio Code (optional)	Code editing
Web Browser	Accessing GitHub and deployed website

4. Theoretical Background

4.1 Software Development Life Cycle (SDLC)

The **Software Development Life Cycle** is a structured process used to develop high-quality software. It ensures that software is built in a systematic and organized manner.

Typical phases of SDLC include:

1. Requirement Analysis
2. System Design
3. Implementation (Coding)
4. Testing
5. Deployment
6. Maintenance

In this lab, we focus on the **Deployment and Maintenance** phases.

4.2 Software Deployment

Deployment is the process of **releasing software to the production environment**, where it becomes accessible to users.

Deployment activities include:

- Uploading application files to a server
- Configuring hosting services
- Managing application versions
- Making the system available to users

In web development, deployment means **publishing a website online** so that it can be accessed using a web browser.

4.3 Software Maintenance

After software is deployed, it must be continuously updated and improved. This process is called **software maintenance**.

Software maintenance includes four major categories:

Corrective Maintenance

Fixing errors or bugs discovered after deployment.

Adaptive Maintenance

Updating the system to work with new environments or technologies.

Perfective Maintenance

Enhancing system functionality or performance.

Preventive Maintenance

Improving system design to prevent future problems.

In websites, maintenance often involves:

- Updating website content
- Fixing broken links
- Updating images or design
- Adding new pages

4.4 Version Control

Version control is a system that tracks changes in software code over time. It allows developers to manage different versions of the same project.

Git is one of the most widely used version control systems.

Advantages of version control:

- Tracks changes in project files
- Allows collaboration among developers
- Enables rollback to previous versions
- Supports project maintenance and updates

4.5 GitHub Platform

****GitHub is a cloud-based platform used for hosting Git repositories.**

Developers use GitHub to:

- Store source code
- Manage project versions
- Collaborate with team members
- Deploy web projects

GitHub also provides **GitHub Pages**, which allows hosting websites directly from a repository.

4.6 GitHub Pages

****GitHub Pages is a static website hosting service provided by GitHub.**

It allows developers to host websites using:

- HTML
- CSS
- JavaScript
- Images and assets

GitHub Pages automatically publishes website files stored in a repository and provides a **public URL**.

Example URL:

`https://username.github.io/project-name`

e

5. Website Project Structure

Before deployment, the website should have a clear folder structure.

Example project structure:

```
website-project
├── index.html
├── about.html
├── contact.html
├── css
│   └── style.css
├── js
│   └── script.js
└── images
```

Important rule:

The main page of the website must be named:

index.html

This page will automatically load when users open the website.

6. Lab Procedure.**Step 1: Create a New Repository**

1. Login to GitHub.
2. Click **New Repository**.
3. Enter repository name: website-deployment-lab
4. Choose **Public** repository.
5. Click **Create Repository**.

The repository will be created.

Step 2: Prepare Existing Website

Students will use an **already developed website project**.

Example files:

index.html
style.css
script.js
images

Ensure that the website works correctly when opened locally.

Step 4: Upload Website Files to GitHub**Method 1 (Using GitHub Interface)**

1. Open your repository.
2. Click **Add File**.
3. Select **Upload Files**.
4. Drag and drop your website files.
5. Click **Commit Changes**.

The files will now appear in the repository.

Step 5: Enable GitHub Pages

1. Open the repository.
2. Click **Settings**.
3. Scroll down to **Pages**.
4. Under **Source**, select:

Deploy from branch

5. Choose:

Branch: main
Folder: /root

6. Click **Save**.

GitHub will begin deploying the website.

Step 6: Access the Deployed Website

GitHub will generate a public website URL:

<https://username.github.io/website-deployment-lab>

b Open this link in your browser.

Your website will now be **live on the internet**.

7. Demonstrating Software Maintenance

After deployment, the website will be updated to simulate **maintenance activities**.

Step 1: Modify Website Content

Open **index.html** and modify the text.

Example:

```
<h1>Welcome to My Website</h1>
```

Change it to:

```
<h1>Welcome to My Updated Website</h1>
```

Save the file.

Step 2: Upload Updated Files

1. Open the GitHub repository.
2. Click **Add File** → **Upload Files**.
3. Upload the modified file.
4. Click **Commit Changes**.

Step 3: Verify Changes

Refresh the deployed website URL.

The updated content will appear on the website.

This demonstrates **software maintenance after deployment**.

8. Expected Output

After completing the lab:

- The website will be successfully deployed online
- Students will receive a public website URL
- The deployed website will load properly in the browser
- Updates made to files will appear on the live website

9. Advantages of Using GitHub Pages

GitHub Pages offers several benefits:

- Free website hosting
- Easy deployment process
- Integration with version control
- Global content delivery network
- Suitable for portfolios and project hosting

It is widely used by developers and students to publish web projects.

EXERCISE

Students are required to take an existing static website project containing HTML, CSS, and related assets and deploy it online using GitHub and its hosting service GitHub Pages. After successful deployment, perform a maintenance activity by modifying some part of the website (such as updating text, changing images, or adding a small section) and upload the updated files to the repository so that the deployed website reflects the changes.

Students must submit the following:

1. GitHub Repository Link
2. Live Website URL
3. Screenshot of the deployed website
4. Screenshot after performing maintenance update